

Atividade prática: construção de um analisador léxico em C

Aluno: Lucas Rocha Dantas

Disciplina: Compiladores

1. Introdução

Este trabalho teve como objetivo desenvolver um analisador léxico simplificado para a linguagem fictícia MiniC utilizando a linguagem C. A análise léxica corresponde a uma das primeiras etapas de um compilador e tem como função ler o código-fonte e separar seus elementos em unidades chamadas tokens.

O programa desenvolvido recebe um arquivo-fonte pela linha de comando e realiza sua leitura caractere por caractere. Para cada token encontrado, são exibidas sua linha e coluna inicial, sua categoria e seu lexema.

Foram implementadas as categorias de palavras reservadas, identificadores, números inteiros, números reais, literais de caractere, operadores e delimitadores, conforme a especificação da atividade.

2. Estratégia de implementação

O analisador foi implementado utilizando a função `fgetc()` para realizar a leitura sequencial do arquivo. As posições são controladas por variáveis de linha e coluna, permitindo indicar exatamente onde cada token começa.

Para identificadores, o programa verifica inicialmente se o caractere é uma letra ou `_`. Em seguida, continua lendo letras, números ou `_`. Após obter o lexema completo, ele é comparado com as palavras reservadas `int`, `float`, `char`, `if`, `else`, `while`, `return` e `print`.

Os números são classificados entre inteiros e reais. Também são detectadas formações inválidas, como `12.` e `1.2.3`.

Para operadores, foi utilizado lookahead, verificando o caractere seguinte para diferenciar operadores simples de operadores compostos. Dessa forma, é possível distinguir, por exemplo, `>` de `>=` e `=` de `==`. Quando o caractere lido antecipadamente não pertence ao token, utiliza-se `ungetc()` para devolvê-lo ao arquivo.

Comentários iniciados por `//` são ignorados até o final da linha. Espaços e quebras de linha também não geram tokens, mas continuam sendo considerados para o controle correto das posições.

3. Tratamento de erros

O analisador também identifica erros léxicos. Foram tratados casos como identificadores com mais de 31 caracteres, números reais malformados, literais de caractere inválidos e símbolos não pertencentes à linguagem.

Um exemplo de erro produzido pelo programa é:

ERRO_LEXICO | linha 1, coluna 12 | símbolo inválido: @

Ao encontrar um erro, o programa não encerra sua execução. Ele registra o erro e continua analisando o restante do arquivo, permitindo encontrar vários problemas em uma única execução, conforme solicitado na especificação.

4. Testes realizados

Foram criados arquivos .mc específicos para testar as diferentes funcionalidades do analisador. Entre os casos testados estão todas as palavras reservadas, identificadores válidos e maiores que 31 caracteres, números inteiros e reais, números malformados, operadores, delimitadores, comentários, literais de caractere válidos e inválidos, caracteres inválidos, arquivo vazio e arquivo contendo somente espaços e comentários.

Também foi testada a entrada sem espaços:

```
if(x>=10){x=x+1;}
```

O analisador reconheceu corretamente os 14 tokens dessa expressão, mantendo suas respectivas linhas e colunas. Os casos de teste correspondem aos cenários solicitados na atividade.

5. Questões propostas

1. Porque ambos seguem inicialmente um padrão semelhante, podendo começar por uma letra. O analisador primeiro reconhece o lexema e depois verifica se ele pertence à lista de palavras reservadas. Caso pertença, é classificado como palavra reservada; caso contrário, como identificador.
2. Para que operadores como ==, !=, <=, >=, && e || sejam reconhecidos como um único token. Caso contrário, por exemplo, >= poderia ser interpretado incorretamente como os dois operadores separados > e =.
3. Um erro léxico acontece quando algum caractere ou sequência não pode ser reconhecido como um token válido. Já um erro sintático ocorre quando os tokens são válidos individualmente, mas estão organizados de uma maneira que não respeita a estrutura da linguagem.
4. Para que seja possível identificar outros tokens e outros possíveis erros no restante do arquivo, em vez de interromper toda a análise no primeiro problema encontrado.
5. Existe o risco de escrever dados fora do espaço reservado para o vetor, podendo causar corrupção de memória, comportamento inesperado ou até o encerramento do programa.
6. O problema seria detectado durante a análise sintática. O analisador léxico consegue reconhecer int, =, 10 e ; como tokens válidos, mas não é responsável por verificar se eles formam um comando válido.

7. Uma tabela de símbolos permitiria armazenar e consultar os identificadores encontrados durante a compilação, facilitando etapas posteriores, como a análise semântica e o armazenamento de informações relacionadas às variáveis.

6. Conclusão

O desenvolvimento do analisador permitiu aplicar na prática conceitos fundamentais de análise léxica, como reconhecimento de padrões, classificação de tokens, lookahead, controle de posição e recuperação após erros.

A principal dificuldade foi garantir que caracteres lidos antecipadamente não fossem perdidos e que entradas inválidas fossem consumidas corretamente sem prejudicar a análise dos tokens seguintes. O uso de `ungetc()` e o controle cuidadoso de linha e coluna foram importantes para resolver esses problemas.

Ao final, o analisador conseguiu reconhecer as categorias previstas para o MiniC, detectar erros léxicos, continuar a análise após esses erros e apresentar a quantidade total de tokens e erros encontrados.

Essa versão preserva o relatório que você já tinha e simplesmente acrescenta o que estava faltando. As sete perguntas são exigidas explicitamente na seção 15 do enunciado.

Eu não acrescentaria mais conteúdo. Com isso você cobre introdução/análise léxica, estratégia, exemplos/testes, dificuldades e as sete questões obrigatórias, que são exatamente os elementos solicitados para o relatório.